

Purpose of the site :

1. Name = Portfolio of Nayant Srivastava
2. Purpose
 - a. For my purpose , it provides a platform for myself to present a personal platform for presenting my projects , career achievements , my works , contributions , progress and my story
 - b. For employer , it provides an open book for the employer to understand what does this candidates have and is it in my forte or not

Requirements :

1. My Section , it tells my story , my past , my current situation , my future plans
2. Project Section , contains everything about my projects
3. Career Section , giving brief and detail of my career
4. Skill Section , skills i can promise
5. Contributions , my contribution to community projects
6. Contact , my contact points

Priortiy in design :

1. Simplicity → Should be not far from a digital interactive resume
2. Direct → It should be direct in stating and explaining the projects , skills etc , not exaggeration , confusing languages , be direct
3. Beauty → Should be beautiful enough to attract someone

About Myself :

I am telling my visitors that i am an AI / ML engineer , specialising in Reinforcement Learning and have broad knowledge of tech like web stack , lang stack , DSA track , system design

Theme :

According to this , the theme should be :

1. Simple
2. Direct
3. Should Focus on a mixture of Maths culture , Coding culture , SDE culture
4. Should not be too much color , we can try tri-color theme at max
5. Should not go for too saturation and totally avoid Neon touch at any place
6. Can experiment Glassmorphism but compare with normal design
7. Try moderate animation , i mean try modernism in animation

System Design of the Site :

Project Type : Web Development and Deployment

Ok we divide our project into multiple ends and consider each end as a separate sub-project
So our portfolio site does not provide much interactive user interface, so this is the division :

1. Frontend → Handles the user end of the site , it includes :
 - a. How the site looks (UI / UX Design)
 - b. How site response (Response Design of the Site)
 - c. How site moves when changes (Animation Design)
2. Backend → Handles the my end of the site , it includes :
 - a. How stores data (Schemas)
 - B . How the site functions on user interaction (Functioning)
 - C. How it process data (Data Streams)
 - D. How it connect to servers and users (API)
3. Data base → Handles the data end of the site , it includes :
 - a. How it stores data (Schemas)
 - B . Query system (DBMS)
4. Server → Handles the hardware end of the site , it includes :
 - A . Choosing server
 - B . Handles the Security etc

1. Frontend

- **Framework: Next.js 15 (App Router) + React 19 + TypeScript.** Latest stable, great for mixing static pages (About, Career) with server components (contributions feed, dynamic project data) in one codebase.
- **Styling: Tailwind CSS v4.** Fast to build a restrained, tri-color, low-saturation system with design tokens (you define exactly 3 hues + neutrals, done).
- **UI primitives: shadcn/ui.** Unstyled, accessible components you fully own and restyle — fits "direct, not templated."
- **Glassmorphism test bed:** build one component (e.g. project card) in both a flat/normal variant and a glass variant using `backdrop-filter`, put them side by side, pick one. Don't commit to glass everywhere — it gets messy on dense text sections like Career/Skills.

2. Animation

- **Framer Motion (Motion for React)** for page transitions, section reveals, hover states — the current standard, well-documented, easy to keep "moderate" (fade/slide/scale, no gimmicks).
- **Lenis** for smooth scroll if you want that modern feel without being heavy-handed.
- Skip GSAP/Three.js unless you specifically want a 3D hero element — for a math/coding/SDE aesthetic, a subtle animated grid, particle-graph, or terminal-typing effect (canvas or SVG) reads better than a 3D scene and is much lighter.

3. Backend

- **Next.js Route Handlers / Server Actions** for contact form submission and any small data processing — no separate backend service needed.

- If you want something closer to a "real" backend for the exercise/learning value (since you're an AI/ML + SDE person and this is also a demo of your skills): **FastAPI (Python)** as a thin API for anything AI/ML-flavored you might showcase (e.g. an interactive RL demo, a project-recommender). This doubles as a skills showcase itself.
- **API layer:** REST is fine and simpler to reason about; use tRPC instead if you stay all-TypeScript and want end-to-end type safety without writing REST boilerplate.

4. Database

- **Sanity.io or Contentful (headless CMS)** if you want to edit project/career/contribution entries without touching code — this is usually the highest-leverage choice for a portfolio, since content changes way more often than layout.
- If you want a real database for the exercise: **PostgreSQL via Supabase** (also gives you auth/storage for free if you ever add a guestbook, analytics, etc.) with **Prisma** as the ORM/schema layer.
- For a portfolio's actual data volume, don't reach for Mongo/NoSQL — your data is relational and small; Postgres is the correct default.

5. Server / Deployment

- **Vercel** — built for Next.js, handles the static + serverless split automatically, free tier is plenty, deploys are instant with zero server management. This is the "latest tech, least friction" answer.
- If you want to own infrastructure as a skills flex: **Docker container on Fly.io or Railway**, fronted by Cloudflare for CDN/DNS/security (WAF, rate limiting) — still minimal ops burden.
- Security essentials regardless of host: HTTPS by default (both platforms above), rate-limit the contact form endpoint, and CSP headers via Next.js middleware.

Suggested "final" stack

Layer	Choice
Frontend	Next.js 15 + React 19 + TypeScript + Tailwind v4 + shadcn/ui
Animation	Framer Motion + Lenis
Backend	Next.js Server Actions (+ optional FastAPI for AI/ML demo)
Database/CM S	Sanity (content) + Postgres/Supabase (if you need real data)
Server/Deploy	Vercel + Cloudflare

This stack is current, all pieces interop cleanly, and — importantly — the frontend does the heavy lifting so you're not maintaining infrastructure for content that rarely changes.

One thing worth deciding before I go further: do you want the CMS (edit content without redeploying) or are you fine hardcoding content in the repo and redeploying on updates? That changes whether "Database" layer 3 is a real requirement or optional.

Color palette :

Type : Tricolor

Color One : #010a0b (Black)

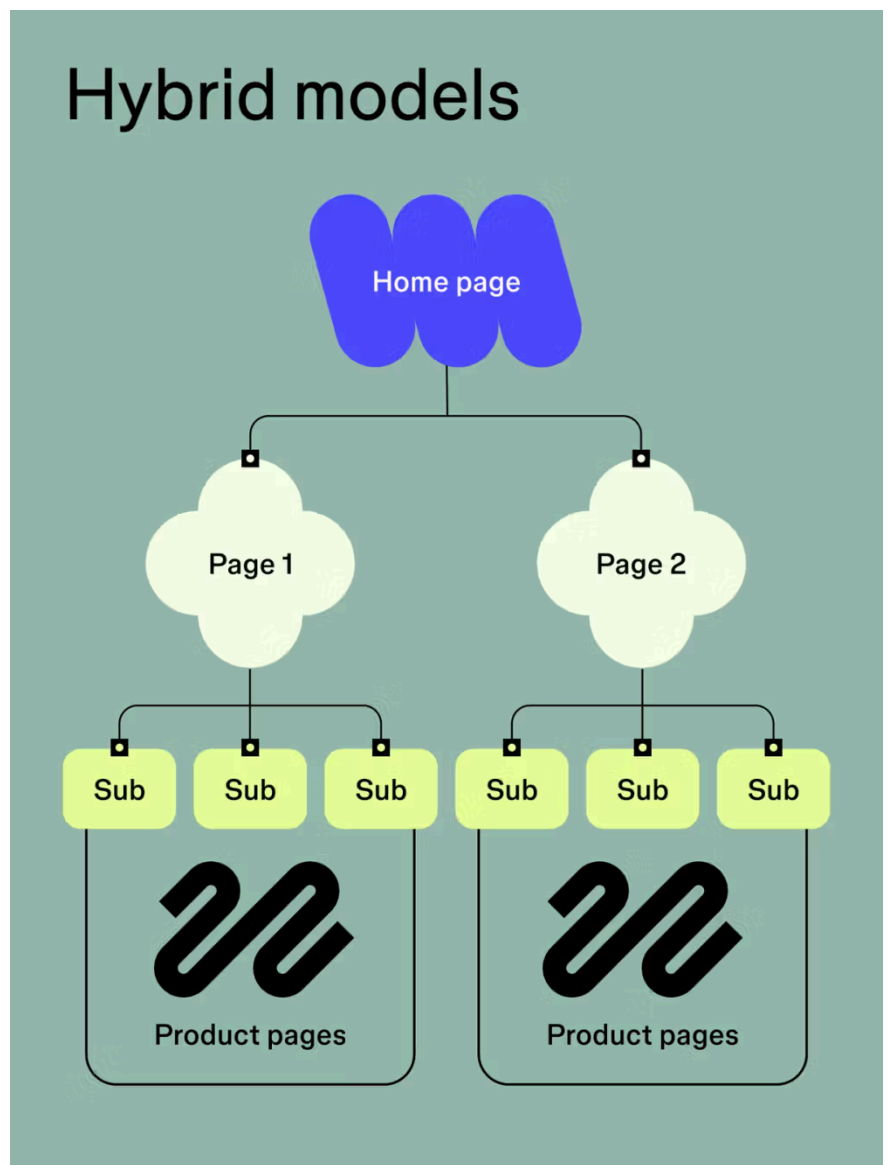
Color Two : #ffffff (White)

Color Three : #f6d009 (Metallic Golden)

Color Ratio :

40 Black : 40 White : 20 Gold

Website Structure :



Trend Inspiration :

1. Collage : Collage Web design brings scrapbook-style creativity into digital experiences. Think sticker graphics, torn textures, cutout photos, and expressive, hand-drawn fonts. It's messy on purpose—and that's what makes it memorable.
2. Lifestyle brands and personal portfolios often use collage layouts to reflect individuality and playfulness. [La Palatine](#), for examples, blends real photos and digital elements in surprising ways.
3. Figma's drag-and-drop tools and endless layer options let you



build, arrange, and remix designs until you find the right look.

Font family :

Aa

ABCDEFGHIJKLMNOPQRSTUVWXYZ

abcdefghijklmnopqrstuvwxyz

1234567890!@#\$%

TYPEFACE

Comfortaa

CATEGORY

Geometric sans serif

COLOPHON

Johan Aakerlund

Comforta

Button Theme :



Something like this :

Nayant Srivastava — Engineering Portfolio

Portfolio Product Specification · v1.0

01 — Product Definition

Product

A personal engineering portfolio designed to present Nayant Srivastava as an **AI/ML engineer with strong software-engineering and systems foundations**.

Primary objective

Convert a visitor's interest into a sufficiently strong understanding of:

1. Who I am
2. What I specialize in
3. What I have built
4. How deeply I understand what I build
5. How I approach engineering problems
6. What evidence supports my claims
7. Where I am going next
8. How to contact me

Core principle

The portfolio is not a gallery of projects. It is evidence of engineering ability.

02 — Target Audience

The site has three primary audiences.

A. Recruiter

Typical attention span:

10–60 seconds

They need to quickly understand:

- Role
- Technical focus
- strongest projects
- experience
- education
- contact
- links

Recruiter question

"Is this candidate relevant enough for me to investigate further?"

B. Technical interviewer / engineer

Typical attention span:

5–20+ minutes

They care about:

- architecture
- implementation
- algorithms
- mathematical reasoning
- engineering decisions
- tradeoffs
- experimentation
- failures
- performance
- results

Interviewer question

"Does this person actually understand what they built?"

C. Fellow engineer / researcher

They may explore the site more deeply.

They care about:

- ideas
- experiments
- technical writing
- research
- implementation details
- interesting problems
- ongoing work

Their question

"Is this person technically interesting?"

03 — Candidate Positioning

The site must establish a clear primary identity.

Primary identity

AI / ML Engineer

Specialization

Reinforcement Learning + intelligent systems

Secondary engineering identity

Software Engineering + Systems

Supporting foundations

- Mathematics
- Statistics
- DSA
- System Design
- Web Engineering

The site should communicate:

Depth first. Breadth second.

Not:

"I know many technologies."

But:

"I have a technical specialty and enough engineering breadth to build complete systems around it."

04 — Homepage Thesis

The homepage should answer this within the first screen:

Who is this person, what does he build, and why should I care?

The hero therefore needs:

Name

Nayant Srivastava

Role

AI / ML Engineer

Supporting statement

Something conceptually similar to:

I build intelligent systems and explore the engineering problems behind them.

The exact copy should be refined later.

Hero actions

Primary:

View Projects

Secondary:

Resume / CV

Optional tertiary:

GitHub

The hero should NOT contain:

- huge technology lists
- 15 badges
- excessive animation

- meaningless motivational statements
 - "Welcome to my portfolio"
 - giant 3D objects that compete with the identity
-

05 — Homepage Information Hierarchy

Recommended order:

HERO

|

|— Identity

|— Specialization

|— Short thesis

|— Primary CTA

|

CURRENT FOCUS

|

SELECTED PROJECTS

|

ENGINEERING PHILOSOPHY

|

EXPERIENCE / JOURNEY

|

CONTRIBUTIONS

|

CURRENTLY BUILDING

|

ABOUT

|

CONTACT

The homepage should be **answer-first**.

Do not make the visitor hunt for your strongest work.

06 — Selected Projects

The homepage should contain only the projects that strengthen the candidate thesis.

Flagship projects

Maximum:

2–3

Each flagship project receives:

- project title
- one-line thesis
- category
- visual
- technical stack
- measurable result where available
- short description
- project status
- CTA

Example information model:

PROJECT

Self-Aware Spacecraft

AI / ML · Anomaly Detection · Intelligent Systems

A system designed to detect abnormal spacecraft telemetry and reason about changing system behavior.

[\[Architecture\]](#) [\[Results\]](#) [\[GitHub\]](#) [\[Demo\]](#)

07 — Project Classification

Every project receives a tier.

S — Flagship

Deep technical case study.

A — Major

Strong project with meaningful technical depth.

B — Experiment

Small project demonstrating exploration or breadth.

Archive

Older / incomplete / experimental work.

This allows the portfolio to grow without every project receiving identical visual importance.

08 — Project Case Study Specification

Every serious project should use a consistent structure.

PROJECT HEADER

Project name
One-line thesis
Status
Timeline
Role
Technologies

[GitHub] [Demo] [Paper / Docs]

01. PROBLEM

What problem exists?

Why is it difficult?

02. MOTIVATION

Why did I build this?

03. APPROACH

What was my proposed solution?

04. SYSTEM

Architecture diagram

Data flow

Component relationships

05. TECHNICAL DEEP DIVE

Algorithms

Mathematics

Implementation

Engineering decisions

06. THE HARD PART

What required the most reasoning?

07. EXPERIMENTS

What did I test?

What hypotheses did I investigate?

08. RESULTS

Metrics

Benchmarks

Comparisons

Screenshots

09. FAILURE

What didn't work?

Why?

What changed?

10. EVOLUTION

v0 → v1 → v2 → current

11. LESSONS

What did the project teach me?

12. FUTURE

What would I improve next?

STACK

LINKS

09 — The "Failure" Principle

A flagship project should not pretend that development was perfect.

A technically mature case study can explicitly show:

Attempt → Failure → Diagnosis → Modification → Result

This is particularly valuable in AI/ML.

For example:

Initial approach



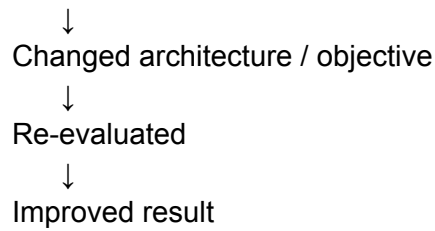
Poor validation performance



Investigated data distribution



Identified failure mode



This turns the project from:

"I used PyTorch."

into:

"I know how to investigate engineering failures."

10 — Evidence System

Every major technical claim should ideally have evidence.

Evidence types

Quantitative

- accuracy
- F1
- recall
- latency
- throughput
- memory
- FPS
- parameter count
- training time
- benchmark results

Engineering

- architecture
- tests
- CI
- deployment
- API
- containerization
- monitoring

Research

- experiment
- baseline
- ablation
- comparison
- hypothesis
- conclusion

Artifact

- GitHub
- demo
- paper
- documentation
- video
- benchmark
- architecture diagram

Rule

Never manufacture impressive numbers for the sake of the portfolio.

If a project has no meaningful quantitative result, present the engineering evidence instead.

11 — About Section

The About section should not be a conventional autobiography.

Use four layers.

Who I am

Professional identity.

How I got here

Short story.

How I think

Engineering philosophy.

Where I'm going

Future direction.

12 — "Now" Section

Add a persistent current-state section.

Example:

NOW

Building

→ Project X

Researching

→ Reinforcement Learning / ...

Learning

→ ...

Contributing

→ ...

Exploring

→ ...

Open to

→ AI/ML / SWE opportunities

This makes the portfolio feel alive rather than archived.

13 — Engineering Philosophy

This section should distinguish the site from a conventional student portfolio.

Possible themes:

Build

Turn ideas into working systems.

Measure

Don't rely on intuition when an experiment can answer the question.

Break

Actively search for failure modes.

Understand

Investigate why something works.

Improve

Iterate based on evidence.

This should eventually become your own philosophy rather than generic motivational copy.

14 — Skills Architecture

Do not use a giant logo wall.

Organize skills around engineering domains.

AI / ML

- Deep Learning
- Reinforcement Learning
- Machine Learning
- PyTorch
- TensorFlow
- etc.

Software Engineering

- Python
- C++
- TypeScript
- React
- Next.js
- FastAPI
- etc.

Systems

- System Design
- APIs

- Docker
- databases
- distributed systems
- etc.

Foundations

- DSA
- Mathematics
- Statistics
- Optimization
- Algorithms

Every major skill should ideally have **project evidence** attached to it.

15 — Career / Experience

Use:

Experience

For formal roles.

Journey

For:

- major projects
- hackathons
- research
- important milestones
- organizations

Contributions

For:

- open source
- communities
- technical contributions
- publications
- events

This prevents the portfolio from feeling empty simply because formal employment history is still developing.

16 — Contributions

The contribution section should answer:

"What does this person contribute beyond his own projects?"

Potential categories:

- Open source
- Hackathons
- Community
- Research
- Technical writing
- Developer tools
- Educational content

Use evidence wherever possible.

17 — Build Log / Lab Notebook

Optional, but highly recommended.

A chronological technical journal:

SEPTEMBER 2026

Experiment:

...

Observation:

...

Problem:

...

Decision:

...

Result:

...

This could eventually become one of the most distinctive parts of the site.

It also creates a mechanism for continuously updating the portfolio without redesigning it.

18 — Visual Design Direction

Primary aesthetic

Technical editorial

Blend:

- mathematics
- engineering documentation
- modern software interfaces
- premium personal portfolio

Avoid becoming:

- cyberpunk
 - gaming UI
 - generic SaaS
 - excessive glassmorphism
 - neon developer portfolio
 - scrapbook-heavy collage
-

19 — Color System

Current proposal:

#010A0B
#FFFFFF
#F6D009

Keep the three-color concept.

However:

Do not enforce 40:40:20 literally.

Instead:

Black / white

→ primary environment

Gold

→ semantic accent

Gold should communicate:

- importance
- interaction
- selection
- achievement
- technical highlight

It should not become the background of large portions of the site.

20 — Typography

Typography should reinforce the technical/editorial identity.

You currently selected **Comfortaa**.

Before committing, test it against:

- long paragraphs
- project descriptions
- numbers
- code
- navigation
- headings
- mobile screens

Do not select a font solely because the heading looks attractive.

You need a typography **system**, not just a font.

Recommended structure:

Display
Heading
Body

The final typeface choices should be made during the design-system phase.

21 — Animation Principles

Animation should have a job.

Good

- hierarchy
- state changes
- navigation
- feedback
- progressive disclosure
- subtle project visualization

Bad

- animation everywhere
- excessive parallax
- long page transitions
- unnecessary 3D
- scroll hijacking
- animations that delay content

Principle

Motion should explain the interface, not compete with the content.

Include:

prefers-reduced-motion

from the beginning.

22 — Responsive Design

Design explicitly for:

Mobile

~360–430px

Tablet

~768–1024px

Desktop

~1280–1440px

Large desktop

1600px+

Do not design desktop first and "make it responsive later."

The project case study especially needs mobile planning because architecture diagrams, tables and technical content can become difficult to read.

23 — Accessibility

Minimum requirements:

- semantic HTML
- keyboard navigation
- visible focus states
- appropriate contrast
- alt text
- reduced-motion support
- accessible buttons
- accessible forms
- logical heading hierarchy

Accessibility should be part of the engineering quality of the site.

24 — Performance

Set explicit goals before implementation.

Prioritize:

- Server Components where appropriate
- minimal client JavaScript
- optimized images
- lazy loading
- font optimization
- code splitting
- caching
- static generation where appropriate

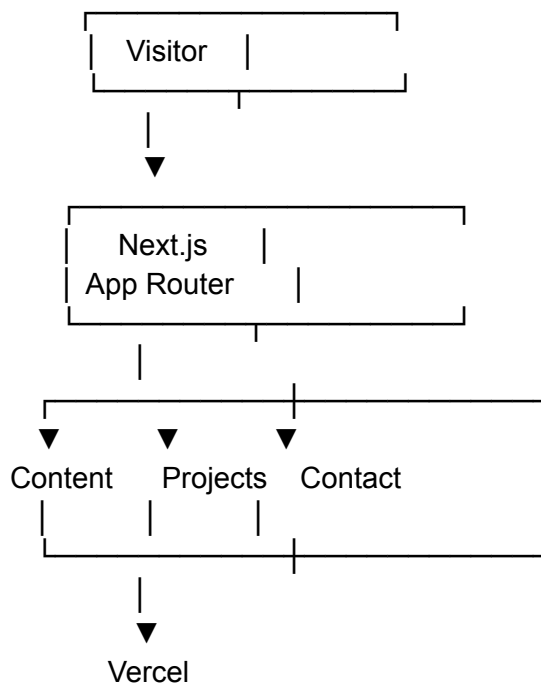
Avoid adding a library simply because it exists.

Every dependency should have a reason.

25 — Technical Architecture

Start simple.

Recommended initial architecture:



Do not introduce FastAPI/PostgreSQL/Sanity unless the product actually needs them.

26 — Content Architecture

Prefer structured project data.

Conceptually:

Project

- |— metadata
- |— summary
- |— problem
- |— motivation
- |— architecture
- |— technicalDeepDive
- |— experiments
- |— results
- |— failures
- |— lessons
- |— futureWork
- |— technologies
- |— links
- |— media

This makes the portfolio scalable.

Adding project #10 should mean:

Add content.

Not:

Rewrite the website.

27 — SEO

Each project should be independently discoverable.

Define:

- title
- description
- Open Graph image
- canonical URL
- structured metadata
- sitemap
- robots
- semantic headings

Project URLs should be stable.

Example:

/projects/self-aware-spacecraft

/projects/animath

/projects/omnix

28 — Analytics

Measure:

Acquisition

- visitors
- referral source
- search traffic

Engagement

- project views
- case-study completion
- GitHub clicks
- resume clicks

Conversion

- contact clicks
- email clicks
- application-related interactions

The most important metric is not:

"How many people visited?"

It is closer to:

"How many relevant visitors reached enough evidence to understand my technical capability?"

29 — Recruiter Validation Test

Before launch, test the homepage with someone unfamiliar with the project.

Give them **10 seconds**.

Ask:

1. What does this person do?
2. What is his strongest area?
3. Name one project.
4. Does he appear technically credible?
5. Would you investigate further?

If the answers are unclear:

fix the information hierarchy before fixing the visuals.

30 — Interviewer Validation Test

Give a technical person a flagship project page.

Ask:

"What questions would you ask this candidate after reading this?"

The ideal result is a list of **technical questions**.

For example:

- Why this architecture?
- Why this algorithm?
- What was the bottleneck?
- How did you evaluate it?
- What failed?
- What would you change?
- What happens at scale?

If they instead say:

"Looks nice."

the case study isn't doing enough technical work.

31 — Trust Rules

The portfolio must follow these rules.

Rule 1

Never exaggerate.

Rule 2

Clearly distinguish:

built by me

from

used / integrated by me

Rule 3

Don't claim research novelty without evidence.

Rule 4

Show limitations.

Rule 5

Prefer evidence over adjectives.

Bad:

"Extremely powerful AI architecture."

Better:

"Implemented a transformer-based pipeline with X components and evaluated it against Y baseline."

32 — Technical Stack

Initial recommendation:

Frontend

Next.js + React + TypeScript

Styling

Tailwind CSS

UI

shadcn/ui selectively

Animation

Motion

Content

MDX / structured local content initially

Backend

Next.js Server Actions / Route Handlers

Deployment

Vercel

CDN / DNS / security

Cloudflare where useful

Database

None initially unless required

Optional future ML service

FastAPI

33 — Design System Before Pages

Before designing individual pages, define:

Foundations

- colors
- typography
- spacing
- grid

- radius
- shadows
- borders
- iconography

Components

- buttons
- navigation
- project cards
- tags
- timeline
- metrics
- code blocks
- architecture diagrams
- links
- accordions
- tabs
- callouts

Motion

- entrance
- hover
- transition
- loading
- reduced motion

Then build pages from these components.

34 — Figma Deliverables

The first Figma phase should contain:

01 — Design tokens

02 — Component library

03 — Desktop homepage

04 — Mobile homepage

05 — Project listing

06 — Flagship project page

07 — About

08 — Experience

09 — Contributions

10 — Contact

11 — Responsive states

12 — Motion prototypes

Do **not** start by designing 15 different pages.

First prove:

Homepage + one flagship case study.

If those work, scale the system.

35 — Definition of Done

The portfolio is not finished when:

"The website looks beautiful."

It is finished when:

Identity

A visitor immediately understands what I do.

Evidence

My strongest projects demonstrate real technical ability.

Depth

An engineer can investigate how I think and build.

Credibility

Claims are supported by evidence.

Navigation

A recruiter can find important information quickly.

Performance

The site is fast.

Accessibility

The site is usable by a broad range of visitors.

Responsiveness

It works properly across devices.

Maintainability

Adding a new project doesn't require redesigning the site.

Conversion

A relevant visitor can easily reach my CV, GitHub, LinkedIn/contact information.

36 — The Core Product Principle

The final design should make the visitor feel:

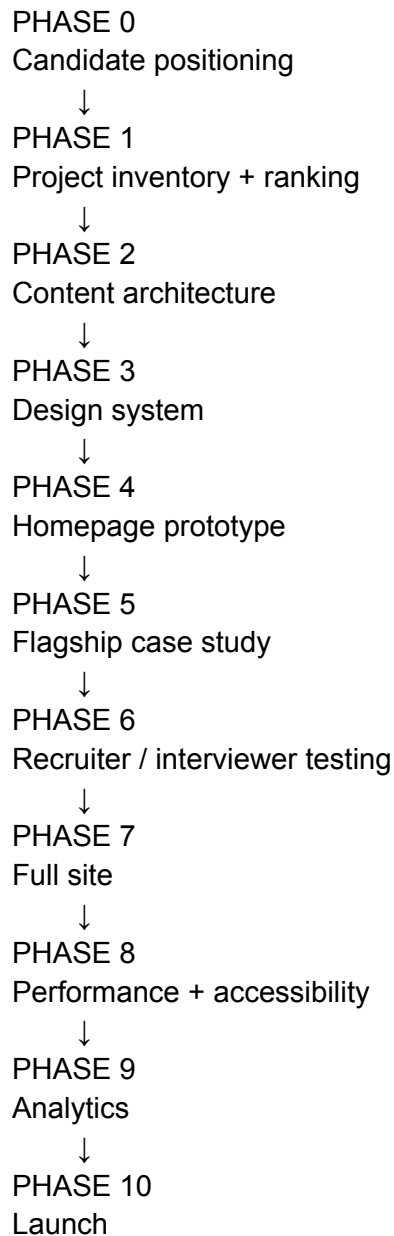
"This person doesn't just know technologies. He builds things, investigates problems, understands the underlying systems, and can explain his decisions."

That is the impression the entire portfolio should optimize for.

37 — Recommended Development Order

Do NOT build the entire website immediately.

Use this sequence:



38 — Immediate Next Actions

Before Figma, complete these **five artifacts**:

Artifact 1 — Candidate Thesis

One paragraph defining exactly who you are professionally.

Artifact 2 — Project Matrix

Every project ranked S/A/B/Archive.

Artifact 3 — Flagship Case Studies

Complete the case-study template for the top 2–3 projects.

Artifact 4 — Content Inventory

Every piece of text/media/link the website will require.

Artifact 5 — Information Architecture

Final sitemap + homepage hierarchy.

Only after these five are complete should visual design begin.